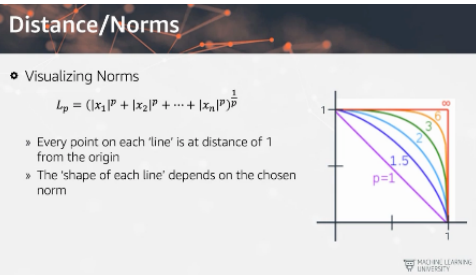
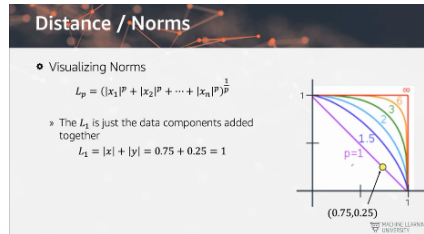


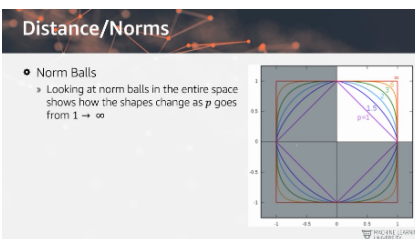
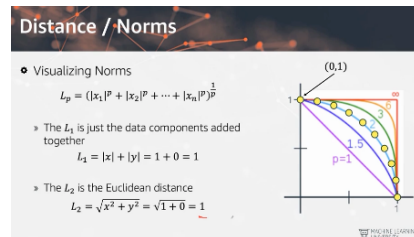
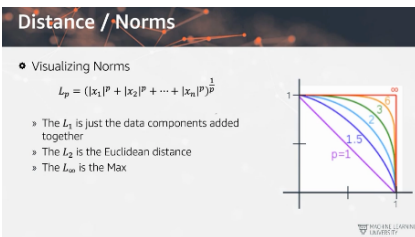
Cada punto de cada línea está a distancia 1 del origen



Para L1 todos los puntos de la línea dan distancia al origen 1

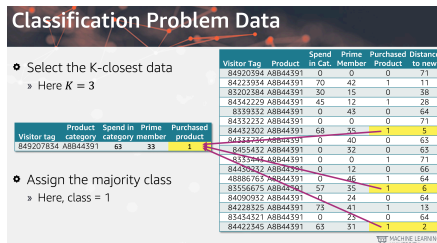
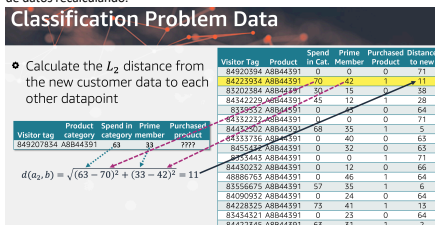


Para L2 todos los puntos de la línea dan distancia 1 también

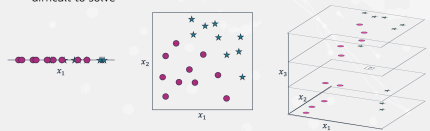


KNN EXAMPLE

es un método no paramétrico ya que no retiene nada de conocimiento con lo que pueda ser predefinido a través de esos parámetros, con cada ejecución tiene que pasar por todos los puntos de datos recalculando.

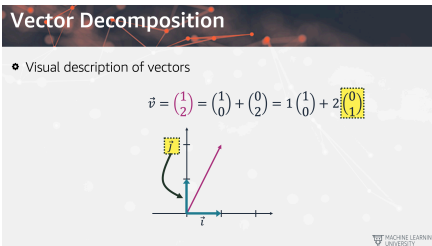


- KNN won't always work
 - With too many features, KNN becomes computationally expensive and difficult to solve



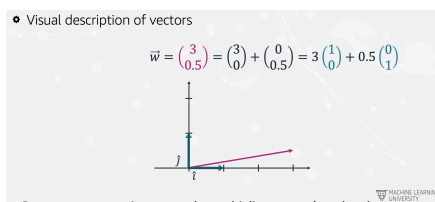
Es muy importante **ESCALAR** los datos en KNN desde el principio, ya que grandes diferencias de magnitud entre ellos hace que baje mucho el rendimiento del algoritmo

DESCOMPOSICIÓN DE VECTORES



La multiplicación de vectores da como resultado otro vector

Other Vector Multiplication Operation



Dos vectores o matrices se pueden multiplicar entre sí resultando En otro vector o matriz del mismo tamaño que el origen

El producto Hadamard puede ser útil para enmascarar matrices

A shape = (3, 5), B shape = (3, 5)

- Vectors can be multiplied by each other
 - » Rethink $4 \times 5 = 20$ in vector form

$\vec{v} = 4\hat{i}, \vec{w} = 5\hat{i}$

» 4×5 is shorthand for adding $\vec{v} = 4$, five times

Hadamard Product

- Element-wise vector & matrix product
 - » Vectors and matrices can be multiplied element-wise when their dimensions are the same

70	0	10	6
15	42	13	4
30	21	8	
45	12	18	5
0	43	13	0
0	9	16	1
68	35	10	1
0	40	9	5
0	32	15	6
0	0	19	4
0	12	14	8
0	46	22	2

0x4	0	0
70x13	42x4	
30x21	15x8	
45x18	12x5	
0	0	0
0	0	0
68x10	35x1	
0	40x5	
0	32x6	
0	0	0
0	12x8	
0	46x2	

0	0	0	0	0
910	168			
630	120			
810	60			
0	0			
0	0			
680	35			
0	200			
0	192			
0	96			
0	92			

```

A.shape = (3, 5); B.shape = (3, 5)

A =
[[0 1 2 3 4]
 [1 2 3 4 5]
 [2 3 4 5 6]]

B =
[[0 1 0 0 1]
 [0 0 1 0 0]
 [0 0 0 1 1]]

A*B.shape = (3, 5)

A*B =
[[0 1 0 0 4]
 [0 0 3 0 0]
 [0 0 0 6 6]]

```

Other Vector Multiplication Operation

La multiplicación de vectores se puede hacer también con Trigonometría

- Vectors can be multiplied by each other
 - » Yes
 - » Requires an extra step to get the components of each vector

$\vec{v} = \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \vec{w} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \vec{w}_1 = \begin{pmatrix} 3 \\ 0 \end{pmatrix}$

Other Vector Multiplication Operation

- Vectors can be multiplied by each other
 - $\vec{v} = \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \vec{w} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \vec{w}_1 = \begin{pmatrix} 3 \\ 0 \end{pmatrix}$
 - $\vec{v} \cdot \vec{w} = 12$

Other Vector Multiplication Operation

PRODUCTO ESCALAR da como resultado un número, no una estructura

- Trigonometric properties
 - » Calculate length of $\text{len}(\vec{w}_1)$
 - » We will use $\cos(\theta) \rightarrow \text{len}(\vec{w}_1) = \text{len}(\vec{w})\cos(\theta)$

$\vec{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}$

$\sin(\theta) = \frac{\text{len}(\vec{w}_2)}{\text{len}(\vec{w})}$

$\cos(\theta) = \frac{\text{len}(\vec{w}_1)}{\text{len}(\vec{w})}$

$\tan(\theta) = \frac{\text{len}(\vec{w}_2)}{\text{len}(\vec{w}_1)}$

$\sin(\theta) = \frac{\text{opp}}{\text{hypot}}$

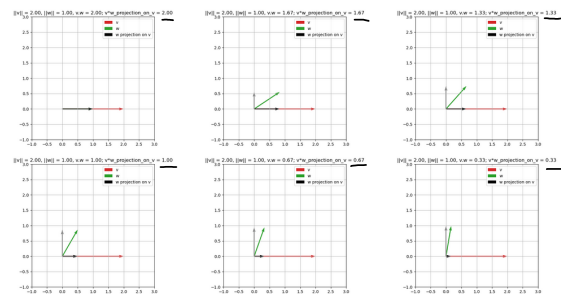
$\cos(\theta) = \frac{\text{adj}}{\text{hypot}}$

$\tan(\theta) = \frac{\text{opp}}{\text{adj}}$

Dot Product

- Cosine Similarity
 - $\cos(\theta) = \frac{\vec{w} \cdot \vec{v}}{\|\vec{w}\| \|\vec{v}\|}$
 - » $\cos \theta$ ranges from -1 to 1
 - » When $\cos \theta = -1$
 - » Vectors are in the exact **opposite direction**
 - » When $\cos \theta = 1$
 - » Vectors are in the exact **same direction**
 - » When $\cos \theta = 0$
 - » Vectors are not similar at all (**orthogonal**)

Cuanto más crezca el ángulo entre los vectores, menor será la proyección de uno sobre el otro y por tanto menor será su producto escalar, y esto hace que se pueda representar claramente su diferencia, por eso se utiliza el producto escalar como una de las herramientas para calcular la similitud entre vectores. Si dejamos fijo uno de los vectores y vamos aumentando el ángulo del otro, el valor decrece



Linear Models

- A Linear Model is a dot product between two vectors
- Linear models are additive functions
 - » Each term in the function has been "weighted"

$$f(x) = x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n$$

weights

Linear Models

- A Linear Model is a dot product between two vectors
- Linear models are additive functions
 - » Each term in the function has been "weighted"

$$f(x) = x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n$$

» Use the dot product to calculate Linear Models

$$f(x) = \vec{x} \cdot \vec{w}$$

Estimation Problem

- The Classification problem with Linear Models
 - » Predict whether a visitor to a gardening product page would purchase at least one gardening product during their visit, based on the amount the customer has spent in the product category and their number of months as a Prime member
 - » Change the problem slightly:
 - » Predict how much a person is going to spend on gardening products during their visit

Estimation Problem

- Looking at a new visitor

Visitor tag	Product category	Spend in category	Prime member	Purchased product	Checkout amount
84920394	AB844391	0	0	0	0
84223534	AB844391	70	42	1	156
83202384	AB844391	30	15	0	0
84342229	AB844391	45	12	1	79
8359352	AB844391	0	43	0	0
84332232	AB844391	0	0	0	0
84492902	AB844391	68	95	1	222
84333756	AB844391	0	40	0	0
845452	AB844391	0	32	0	0
8354443	AB844391	0	0	1	44
84502352	AB844391	0	12	0	0
48886763	AB844391	0	46	1	234
83556675	AB844391	57	35	1	255
84000532	AB844391	0	24	0	0
84228325	AB844391	73	41	1	204
8341421	AB844391	0	23	0	0
84567549	AB844391	0	9	1	15

$$\vec{x} = \begin{pmatrix} 63 \\ 33 \end{pmatrix}, \quad \vec{w} = \begin{pmatrix} 2.5 \\ 0.8 \end{pmatrix}$$

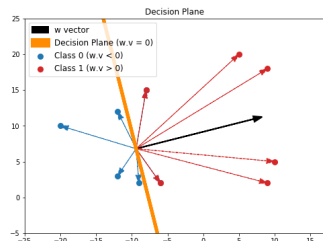
$$63 \cdot 2.5 + 33 \cdot 0.8 \approx \$186$$

El modelo más simple de un Modelo Lineal utilizando el producto escalar entre vectores es el Decision Plane, que en su forma más simple (plano ortogonal) permite decidir entre dos opciones creando un Plano vectorial que separa ambas opciones. Se trata de encontrar la línea del plano que al multiplicar la variable X por el peso W si es mayor que 0 será positivo y si es menor negativo (en este caso es COMPRA o NO_COMPRA)

Decision Plane

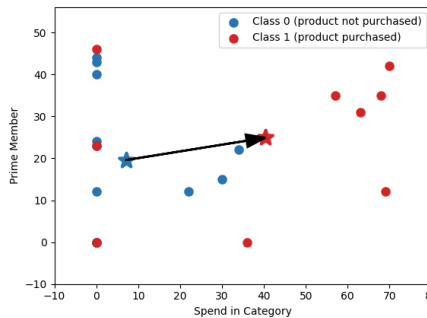
This storyline gives us an even easier algorithm to make class predictions from just the sign of dot products. For example, to solve a binary classification task, we would pick a vector $\vec{w}$ pointing towards say, class 1; all the (training data points) vectors $\vec{v}$ such that $\vec{w} \cdot \vec{v} > 0$ would be pointing in roughly the same direction as $\vec{w}$, and thus be assigned to class 1; whereas all the vectors $\vec{v}$ such that $\vec{w} \cdot \vec{v} < 0$ would be pointing in the opposite direction, and thus be assigned to the other class, class 0.

Most importantly, from a geometric point of view, this is also to say that, given a vector $\vec{w}$ - the black vector in the figure below, all vectors $\vec{v}$ such that the dot product $\vec{v} \cdot \vec{w} = 0$, are orthogonal on the vector $\vec{w}$. The heads of all these vectors $\vec{v}$ determine a so-called hyperplane orthogonal on the vector $\vec{w}$ - here a line, the orange line depicted below. Furthermore, all vectors $\vec{v}$ such that $\vec{v} \cdot \vec{w} > 0$ are pretty much pointing in the same general direction with the vector $\vec{w}$, on one side of the hyperplane (the red vectors depicted below), whereas all vectors $\vec{v}$ such that $\vec{v} \cdot \vec{w} < 0$ are pretty much pointing in the opposite direction of the vector $\vec{w}$, on the other side of the hyperplane (the blue vectors depicted below). And so, this hyperplane described by a dot product, splits the data space into two subspaces, each subspace corresponding to distinct classes associated to some particular values of the dot product with respect to a threshold. We can think of this hyperplane as a binary classifier that separates the two classes. In this context, such hyperplane is often referred to as a decision plane.



En resumen, los pasos para generar un plano de decisión binario son:

- * Computar la media de los vectores `ave_0` y `ave_1` asociados a las clases 0 y 1 respectivamente con el dataset de training `X_train`.
- * Usando el vector `W` resultante de conectar las dos medias de las clases, computar el producto escalar con todo el dataset `X_test`.
- * Hacer predicciones sobre `X_test`, probando los distintos umbrales (en lugar de hacer mayor/menor que 0, hacerlo sobre 1,2,3,-1,-2,-3,...) sobre el producto vectorial recogiendo cual de ellos da mejor rendimiento



Se puede observar que prácticamente todos los puntos rojos están en la misma dirección que el vector $\vec{W}$ (producto escalar positivo) mientras que la mayoría de los azules tienen dirección opuesta -1 a $\vec{W}$

Es posible que el umbral 0 no nos brinde el mejor plano de decisión (ortogonal en el vector de peso), pero diferentes opciones del umbral (que corresponden visualmente a mover la línea ortogonal hacia la izquierda o hacia la derecha) deberían mejorar el clasificador.

Tendríamos que iterar para buscar el valor del umbral y que fuera en lugar de mayor/menor que 0, hacerlo con +1,+2,+3, etc,

KNN vs Decision Plane for binary classification

Al evaluar los distintos modelos como puede ser KNN que es un método no paramétrico que no aprende nada ya que calcula todas las distancias del dataset al punto a predecir frente a métodos como el plano de decisión en el que sí aprendemos porque tomamos la media de todos nuestros puntos pudiéndole aplicar un threshold, con lo que no hace falta volver a pasar por todos los puntos del dataset para predecir, existen habitualmente 3 maneras de evaluar:

MSE (Mean Squared Error) -> Media de las Distancias absolutas entre valor y predicción (para que no salgan distancias negativas hay que ponerlo al cuadrado)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

RMS (Root Mean Squared) -> Raíz cuadrada para que sea una distancia entendible, ya que el anterior hay que ponerlo al cuadrado para que no salgan distancias negativas.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

R2 -> No da distancia absoluta sino cuanto de la variación es explicada por el modelo

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$